Experiment No.: 04
Title: SVM Classification Algorithm

Batch: B3 **Roll No.: 16014225801** **Experiment No.: 04**

Aim: Design and implement of SVM classification algorithm

Resources needed: Python

Theory:

The Support Vector Machine (SVM) is a Supervised Learning algorithm. It is powerful and widely used for classification and regression tasks in machine learning. It's particularly effective for binary classification tasks, where the goal is to separate data points of two different classes by finding an optimal hyperplane that maximises the margin between them.

SVM Classification Algorithm:

- **Data Preparation:** Prepare your labelled training data. Each data point should have a set of features (attributes) and a corresponding class label indicating its category (e.g., 0 or 1, A or B).
- **Feature Scaling:** It's essential to scale the features so that they have a similar range. Common scaling techniques include normalization or standardization.
- **Kernel Selection:** Choose a kernel function to transform the data into a higher-dimensional space. SVM uses the kernel trick to find non-linear decision boundaries. Commonly used kernels are:
- **Linear Kernel:** For linearly separable data (the default kernel when no other kernel is specified).
- **Polynomial Kernel:** Suitable for data with polynomial patterns.
- **Radial Basis Function (RBF) Kernel (Gaussian Kernel):** Suitable for non-linear data and is the most popular kernel choice.
- **Sigmoid Kernel:** Useful for neural network-inspired architectures.
- **Parameter Selection:** SVM has two crucial parameters to be set:
- **C (Cost parameter):** Controls the trade-off between maximizing the margin and minimizing the classification error on the training data. A smaller C allows for a wider margin but may result in misclassifying some training examples, while a larger C aims to classify all examples correctly, potentially leading to a narrower margin.
- **Kernel-specific parameters (e.g., gamma for RBF kernel):** The effect and interpretation of these parameters depend on the selected kernel. Proper tuning is essential for optimal performance.
- **Model Training:** Feed the scaled and transformed data into the SVM algorithm, which optimizes the position of the hyperplane to achieve the best separation between classes. This process involves solving a convex optimization problem to find the optimal weight vector and bias term that define the hyperplane.

Procedure:

1. Identify attributes suitable for applying classification algorithm
2. Implement SVM on your dataset using Python and RapidMiner.
3. Apply SVM to classify unknown tuple.

Results:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

# Step 1: Load Data
df = pd.read_csv('response_1786094863508.csv')

# Step 2: Attribute Selection and Target Label Creation
# Create binary class label based on median ANNUAL temperature
median_temp = df['ANNUAL'].median()
df['Label'] = (df['ANNUAL'] > median_temp).astype(int)

# Define feature matrix X and target vector y
feature_cols = ['JAN-FEB', 'MAR-MAY', 'JUN-SEP', 'OCT-DEC']
X = df[feature_cols]
y = df['Label']

# Step 3: Feature Scaling (Standardization)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 4: Train SVM Model (RBF Kernel)
svm_model = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
svm_model.fit(X_scaled, y)

# Step 5: Classify an Unknown Tuple
# Sample unknown observation: [JAN-FEB=19.10, MAR-MAY=26.20, JUN-SEP=27.25, OCT-
DEC=21.80]
unknown_tuple = pd.DataFrame([[19.10, 26.20, 27.25, 21.80]], columns=feature_cols)

# Scale unknown tuple using the fitted scaler
unknown_scaled = scaler.transform(unknown_tuple)

# Predict class
prediction = svm_model.predict(unknown_scaled)

# Display Results
print(f"Median Annual Temp Threshold: {median_temp:.2f}°C")
print(f"Unknown Input Tuple: {unknown_tuple.iloc[0].to_dict()}")
print(f"Predicted Class Label: {prediction[0]} ({'Above Average Temp Year' if
prediction[0] == 1 else 'Below Average Temp Year'})")
```

Outputs

```
Median Annual Temp Threshold: 23.87°C
Unknown Input Tuple: {'JAN-FEB': 19.1, 'MAR-MAY': 26.2, 'JUN-SEP': 27.25, 'OCT-DEC':
21.8}
Predicted Class Label: 1 (Above Average Temp Year)
```

Questions

1. Explain over fitting problem in Machine Learning.
2. What are advantages and disadvantages of SVM?

Answers

1. Overfitting in Machine Learning

Overfitting occurs when a machine learning model learns both the underlying patterns and the random noise in its training data too thoroughly. As a direct result, the algorithm essentially memorizes the training set rather than generalizing from it. While this leads to exceptionally high accuracy on training data, the model fails significantly when evaluated on new, unseen test data, resulting in high variance, poor generalizability, and highly unreliable future predictions.

2. Advantages and Disadvantages of SVM

Support Vector Machines excel in high dimensional spaces, offer strong memory efficiency by relying only on key support vectors, and handle complex nonlinear boundaries effectively using kernel tricks. However, SVMs suffer from notable disadvantages: they are computationally expensive and slow to train on large datasets, highly sensitive to noise and outliers, require careful feature scaling, and do not provide direct probability estimates, making model output interpretation challenging without additional calibration.

Conclusion:

The Support Vector Machine classification algorithm was successfully designed, implemented, and evaluated using Python. Proper feature scaling and RBF kernel transformation effectively enabled accurate model training and reliable classification for an unknown dataset tuple.

References:

1. Stephen Marsland, “Machine learning An Algorithmic Perspective CRC”, Press, 2015
2. Christopher M. Bishop “Pattern Recognition and Machine Learning Springer Science, Business Media 2006
3. Simon Haykin, “Neural Networks and Learning Machines” Pearson Education, 2009